

Словарик терминов.

Слова, которые постоянно всплывают в разборах и на собеседованиях. У каждого: что значит, откуда взялось и где встречается в проекте.

Файл пополняется по мере разбора задач — как и `heuristics.md`, только там приёмы, а здесь термины.

Характеристики решения — ранний выход, работа на месте, устойчивость, детерминированность и прочие строки таблицы «Свойства решения» — вынесены в отдельный файл `algorithm-properties.md`.

О-нотация: $O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Способ записать, как растёт стоимость алгоритма при росте входа.

Читается «о большое от...». Внутри скобок — не число операций, а закон роста: во сколько раз подорожает работа, если вход станет вдвое, втрое, вдесятеро больше.

Запись	Как называется	Что происходит при удвоении входа	Пример
$O(1)$	константное	ничего, цена та же	взять <code>nums[5]</code> , положить в <code>HashSet</code>
$O(\log n)$	логарифмическое	прибавляется один шаг	бинарный поиск
$O(n)$	линейное	вдвое дороже	один проход по массиву
$O(n \log n)$	линейно-логарифмическое	чуть больше чем вдвое	сортировка
$O(n^2)$	квадратичное	вчетверо дороже	перебор всех пар вложенным циклом

Главная формула, вокруг которой всё крутится:

сложность = сколько шагов × сколько стоит один шаг.

Считать надо обе величины. Алгоритм из n шагов по $O(1)$ — это $O(n)$; из n шагов, каждый по $O(n)$, — уже $O(n^2)$.

Две ходовые путаницы:

- **$O(1)$ не значит «быстро» и не значит «одна операция».** Внутри может быть десяток действий — важно лишь, что их число не растёт вместе с длиной входа.
- **O не про точное время.** Константы и слагаемые помельче отбрасываются: $2n + 100$ — это $O(n)$. Поэтому два решения с одинаковой оценкой могут различаться по скорости в разы, и на собеседовании про это уместно сказать отдельно.

Где у нас: `LongestSubstringNoRepeat.md`, раздел «Почему это $O(n)$ », несмотря на вложенный `while` — там формула разобрана на конкретных строках кода.

У какого логарифма основание в $O(\log n)$.

У любого. Основание внутри O не пишут, потому что оно ни на что не влияет: логарифмы разных оснований отличаются постоянным множителем, а постоянные множители O отбрасывает.

$$\begin{aligned}\log_2 n &= \log_{10} n / \log_{10} 2 \approx \text{примерно равно } 3,32 \cdot \log_{10} n. \\ \log_2 n &= \ln n / \ln 2 \approx \text{примерно равно } 1,44 \cdot \ln n.\end{aligned}$$

Множитель не зависит от n , поэтому $O(\log_2 n)$, $O(\log_{10} n)$ и $O(\ln n)$ — одна и та же оценка.

А вот в **фактическом числе шагов** основание живёт, и задаёт его алгоритм — на сколько частей он делит задачу за один шаг:

Алгоритм	Делит на	Шагов при $n = 1000$
бинарный поиск.	2.	$\approx$ примерно равно 10.
тернарный поиск.	3.	$\approx$ примерно равно 7.
поиск в В-дереве с ветвлением 100.	100.	$\approx$ примерно равно 2.

Все три — $O(\log n)$, а по числу шагов различаются в разы.

Отсюда оговорка к строке таблицы выше: «при удвоении входа прибавляется один шаг» верно для основания 2, то есть для деления пополам. У тернарного поиска шаг прибавляется при утроении.

В алгоритмических текстах основание по умолчанию подразумевают двойкой — просто потому, что задачи чаще делятся пополам. Это соглашение о чтении, а не часть определения.

Инвариант.

Утверждение, которое остаётся истинным на каждом шаге алгоритма, хотя сами переменные при этом меняются.

Слово прячет смысл внутри себя: `in-` (не) + `varias` (меняющийся), тот же корень, что у «варьировать» и «вариант». Инвариант — «то, что не варьируется».

Частая путаница: кажется, что инвариант — неизменная величина. Наоборот: величины как раз меняются, неизменной остаётся **правда о них**.

Зачем нужен: это способ доказать корректность, не проходя алгоритм в голове шаг за шагом. Проверяются три вещи — инвариант верен в начале, каждый шаг его сохраняет, и в конце из него следует нужный ответ. Сломанный инвариант обычно и есть баг.

Где у нас:

- «в окне нет повторов» — `LongestSubstringNoRepeat.md`;
- «левая граница только растёт» — там же, из-за него в варианте с прыжком нужен `Math.max`;
- «в момент записи `writeIndex <= readIndex`» — `MoveZeroes.md`;
- «ширина окна ровно `k`» — `MaxAverageSubarray.md`;
- «в стеке лежат незакрытые скобки в порядке появления» — `ValidParentheses.md`.

Осторожно с омонимом: в Java-дженериках «инвариантность» значит совсем другое — что `List<String>` не подтип `List<Object>`.

Амортизированная сложность.

Средняя стоимость операции, если растянуть редкие дорогие шаги на всю последовательность.

Не то же самое, что «в среднем случае». Средний случай — про удачные и неудачные входные данные, амортизация — про то, что дорогой шаг случается редко и его цена размазывается по дешёвым.

Классика: добавление в `ArrayDeque` или `ArrayList` стоит $O(1)$ амортизированно. Изредка массив надо расширить, и эта конкретная вставка обойдётся в $O(n)$, но расширения происходят всё реже по мере роста.

Где у нас: `Queue.md` — про рост массива; `Pattern_HashMap.md` — почему у хеш-таблицы корректно говорить «амортизированное $O(1)$ », а не «строго $O(1)$ ».

Амортизационный анализ.

Способ считать сложность по суммарной работе, а не по вложенности циклов.

Родственник предыдущего термина, но применяется к другому случаю: когда внутренний цикл выглядит опасно, а на деле алгоритм целиком линеен.

Главное в этом способе — что именно суммируется. Считается работа **за все итерации внешнего цикла вместе**, а не худшая итерация, помноженная на их число. Второй способ даёт завышенную оценку: внутренний цикл может сделать много шагов на одной итерации, но тогда на других он не сделает почти ничего.

Опорный пример — скользящее окно: внутри `for` стоит `while`, но левая граница движется только вперёд и за всё время работы проходит строку один раз. Всего не больше $2n$ сдвигов, значит $O(n)$, несмотря на вложенность.

Где у нас: `Pattern_SlidingWindow.md`, раздел «Почему обе границы вместе дают $O(n)$ ».

Фолбэк.

Ветка, которая срабатывает, когда цикл закончился, ничего не найдя.

От английского `fall back` — «отступить, откатиться к запасному варианту». В коде это обычно `return` ПОСЛЕ цикла: `return -1`, `return false`, `return {-1, -1}`.

Слово жаргонное, а не терминологическое: в своих заметках так говорить удобно, а вслух на собеседовании лучше «ветка после цикла» или «запасной ответ».

Почему важно: это самая частая ветка, на которую забывают написать тест. Цикл с находкой внутри тестируется легко, а случай «перебрали всё впустую» ускользает.

Где у нас: `heuristics.md`, приём 3 — там же разобрано, когда `return` идёт внутрь цикла, а когда после него.

Предусловие.

Требование к входу, которое метод не проверяет, а считает выполненным.

По-английски `precondition`. Приставка «пред-» тут буквальная: условие, которое должно быть истинным ДО вызова. Обеспечивает его вызывающая сторона, а не сам метод.

Зачем так делают: проверка иногда стоит дороже самой работы. Тогда её выносят в договорённость и записывают в документацию метода.

Чем отличается от охранной проверки: охранный проверка — это то, что метод контролирует сам и на чём бросает исключение (`null`, отрицательная длина). Предусловие он не контролирует вовсе, и при его нарушении не падает, а молча возвращает неверный ответ. Второе опаснее, поэтому предусловие принято проговаривать вслух.

Где у нас: `BinarySearch.md` — отсортированность массива; проверить её стоило бы `O(n)`, а весь алгоритм работает за `O(log n)`.

Отпечаток (ключ группировки).

Величина, одинаковая у всех элементов одной группы и разная у элементов из разных.

Английское `fingerprint` или просто `key`. Позволяет заменить попарное сравнение на раскладывание по корзинам за $O(1)$.

Устоявшегося русского термина у этой величины нет: в разборах задачи пишут описательно — «отсортированные буквы», «ключ словаря». «Отпечаток» здесь рабочее название, своё для этого проекта, и на собеседовании его надо расшифровывать, а не произносить как общеизвестное.

Главная ловушка — склейка: отпечаток обязан не только совпадать у эквивалентных, но и **различаться** у неэквивалентных.

Где у нас: `GroupAnagrams.md` — отсортированные буквы и вектор частот как два разных отпечатка; `heuristics.md`, приём 7 — про проверку на склейку.

Кодовая точка и суррогатная пара.

Кодовая точка — номер символа в Unicode. **Суррогатная пара** — два значения `char`, которыми Java записывает один символ, не влезающий в 16 бит.

Практический вывод: `char` — это не «символ», а 16-битная ячейка. Эмодзи занимает два `char`, и посимвольный обход разваливает его на половинки.

Где у нас: `String.md` — устройство; `ValidAnagram.md` и `FirstUniqueChar.md` — решения по кодовым точкам вместо `char`.

LIFO и FIFO.

LIFO (last in, first out) — «последним пришёл, первым вышел», поведение стека. **FIFO** (first in, first out) — «первым пришёл, первым вышел», поведение очереди.

Где у нас: `Stack.md`, `Queue.md`. Практическая разница видна в обходах: на очереди BFS даёт кратчайший путь по числу рёбер, на стеке это свойство теряется.

Подстрока и подпоследовательность.

Подстрока — отрезок: позиции подряд. **Подпоследовательность** — набор: позиции в возрастающем порядке, но между ними могут оставаться невзятые.

Различие решает, применим ли приём «скользящее окно»: окно умеет только подстроки.

Где у нас: подробный разбор с ASCII-схемами — в `LongestSubstringNoRepeat.md`, раздел «Подстрока и подпоследовательность».

`[x]` и `[x]` — округление вниз и вверх.

`[x]` — округление вниз (`floor`, пол), `[x]` — округление вверх (`ceiling`, потолок).

Запись.	Название.	Что делает.	Пример.
<code>[x]</code> .	<code>floor</code> , пол.	к меньшему.	<code>[9,97] = 9</code> .
<code>[x]</code> .	<code>ceiling</code> , потолок.	к большему.	<code>[9,97] = 10</code> .

Метафора сидит в самих названиях: пол внизу, потолок наверху. Начертание тоже подсказывает — у `[` чёрточки смотрят вниз, у `]` вверх.

Осторожно с отрицательными: `floor` округляет **к минус бесконечности**, а не «отбрасывает дробную часть». `[-2,5] = -3`, тогда как целочисленное деление в Java усекает в сторону нуля и даёт `-2`. Разбор — в `Rounding.md`.

Где у нас: `BinarySearch.md` — число итераций записано как `[log2 n] + 1`: на тысяче элементов `log2 1000 ≈ примерно равно 9,97`, пол даёт девять, плюс единица — десять.

even и odd — чётный и нечётный.

even — чётный, **odd** — нечётный. Пара путается, поэтому запоминать её лучше не переводом, а вторыми значениями этих слов: они не случайные, а прямо про чётность.

even — **ровный**: **even surface** — ровная поверхность, **even split** — деление поровну. Чётное число как раз то, которое делится **ровно** пополам, без остатка.

odd — **странный, лишний, непарный**: **odd socks** — носки без пары, **odd one out** — лишний в ряду. Нечётное число — ровно это: разбили на пары, один остался лишним.

even	→ соответствует	● ●	● ●	● ●	все разбились на пары.
odd	→ соответствует	● ●	● ●	●	один лишний, без пары.

Запасной костыль, если смысловая связка не вспомнилась: **odd** — три буквы, число нечётное; **even** — четыре, число чётное.

Где у нас: **BinarySearch.md** — в тестах класса массив **evenNumbers** заполнен чётными числами, а нечётные подставляются как значения, которых в массиве заведомо нет, и проверяют фолбэк **-1**.

Definite assignment.

Требование Java: переменная обязана быть гарантированно присвоена до первого чтения.

Компилятор рассуждает формально: если присваивание сидит внутри **if** внутри цикла, который теоретически может не начаться, — значит гарантии нет, и код не соберётся.

Где у нас: **MajorityElement.md** — из-за этого правила кандидат инициализируется явно, а не остаётся пустым.

Поддерево.

Узел вместе со всем, что растёт из него вниз, — потомки, потомки потомков и так далее до листьев.

Поддерево само является деревом: у него есть корень (выбранный узел), и к нему применимо всё, что применимо к целому дереву — глубина, обход, поиск. На этом держится рекурсия по деревьям: вопрос, заданный дереву, задаётся каждому поддереву, и ответы складываются.

У узла двоичного дерева поддерева два: левое растёт из поля `left`, правое из `right`. Если потомка нет, поддерево пустое — в коде это `null`.

```
      3           поддерева корня 3:
     / \         левое — узел 9 (потомков нет).
    9   20        правое — 20 с потомками 15 и 7.
   /  \
  15   7         поддерева узла 9: оба пустые.
```

Не путать с частью дерева, вырезанной произвольно: поддерево всегда берётся целиком, до самого низа. Узел `20` без `15` и `7` поддеревом не является.

Где у нас: `MaxDepthBinaryTree.md` — глубина дерева считается через глубину его двух поддеревьев. Справочник с картинкой — `BinaryTree.md`.

Прямой, симметричный и обратный порядок обхода (pre-order, in-order, post-order).

Три способа обойти двоичное дерево в глубину; отличаются тем, в какой момент узел считается посещённым.

Рекурсивный обход проходит мимо каждого узла трижды: при входе, между левым и правым поддеревом и при возврате. Отсюда три порядка:

Порядок	Английское название	Когда узел посещается	На дереве [1, 2, 3, 4, 5, 6, 7]
прямой	pre-order	при входе, до поддеревьев	1, 2, 4, 5, 3, 6, 7
симметричный	in-order	между левым и правым поддеревом	4, 2, 5, 1, 6, 3, 7
обратный	post-order	при возврате, после обоих поддеревьев	4, 5, 2, 6, 7, 3, 1

Приставки pre-, in-, post- говорят о положении узла относительно его поддеревьев: до, между, после. Код у всех трёх один, отличается место строки «посетить узел».

Где у нас: `BinaryTree.md` — картинки всех трёх порядков и обхода в ширину; `MaxDepthBinaryTree.md` — обратный порядок в деле: высота считается после того, как ответили оба поддерева.

Обход (traversal)

Посещение всех узлов структуры по одному, в определённом порядке.

Английское traversal — от глагола traverse, «пройти насквозь, пересечь». Слово не привязано к деревьям: обходят граф, список, каталоги на диске. У деревьев обход бывает в глубину (прямой, симметричный и обратный порядок) и в ширину (по уровням); отличаются они моментом, когда узел считается посещённым, и структурой под рукой — стек или очередь.

Где у нас: `BinaryTree.md` — все четыре обхода с картинками; `LevelOrderTraversal.md` — обход по уровням как задача.

Двоичное дерево поиска (binary search tree, BST)

Двоичное дерево с правилом порядка: у каждого узла всё в левом поддереве меньше его значения, всё в правом — больше.

Правило действует на всё поддерево, а не только на прямых потомков. Из него следуют поиск спуском по одной ветке за $O(h)$ и

отсортированный вывод при симметричном обходе. Слово «поиска» — потому что структура заведена ради поиска: каждое сравнение отбрасывает целое поддерево, как бинарный поиск отбрасывает половину массива.

Сбалансированность в определение не входит: отсортированная вставка даёт цепочку, которая остаётся деревом поиска, но ищет за $O(n)$.

Где у нас: `BinarySearchTree.md` — определение, картинки, цепочка против сбалансированного дерева, сборка из отсортированного массива.

Сбалансированное дерево.

Дерево, у которого в каждом узле высоты левого и правого поддеревьев различаются не больше чем на единицу.

Смысл — высота $h \approx \text{примерно равно } \log_2 n$, и всё, что стоит $O(h)$ (поиск, вставка, глубина рекурсии), становится логарифмическим. Противоположный край — цепочка, $h = n$. Свойство независимо от правила порядка: бывает сбалансированное дерево без порядка и дерево поиска без баланса. Деревья, которые сами держат баланс при вставках и удалениях, — красно-чёрное (`TreeMap`, `TreeSet`), AVL.

Где у нас: `BinarySearchTree.md`, рис. 2 — цепочка и сбалансированное дерево из одних значений; Макдауэлл, задача 4.4 «Проверка сбалансированности».

`unless` — «если только не», «кроме случая, когда»

Союз-отрицание: `A unless B` значит «A, если не B». Равносильно `A if not B`.

В именах методов и в условиях читается как исключение из правила: `retry unless cancelled` — «повторять, если только не отменено»; `enqueuePairUnlessMismatch` — «положить пару в очередь, если только нет несовпадения». Ловушка при чтении: `unless` уже содержит «не», и второе отрицание после него переворачивает смысл — `unless not found` значит «если найдено».

Где у нас: обсуждение имени `pushPairOrFail` в `SymmetricTreeIterative` — вариант `enqueuePairUnlessMismatch` отвергнут, потому что у метода три исхода, а «unless mismatch» покрывает только два.

Двоичная куча (binary heap).

Законченное двоичное дерево, в котором значение каждого узла не больше значений его потомков. Хранится массивом, без ссылок на потомков.

Слово «куча» тут не про кучу объектов в памяти (heap как область JVM) — это независимые омонимы, и на собеседовании их иногда путают. Свойство кучи слабее свойства дерева поиска: оно связывает узел только с прямыми потомками, а порядка между соседними ветвями нет, поэтому обход кучи ничего не сортирует. Зато законченность позволяет хранить дерево массивом: потомки узла с индексом i лежат на $2i + 1$ и $2i + 2$, родитель — на $(i - \text{минус } 1) / 2$.

Где у нас: `PriorityQueue.md` — устройство, подъём и спуск, применение; `TreeNotation.md` — та же арифметика индексов как «полное представление» дерева.

Законченное, полное и идеальное дерево.

Три разных слова про форму двоичного дерева. Путаются постоянно, потому что по-русски звучат похоже, а означают разное.

Законченное (complete) — каждый уровень, кроме последнего, содержит полный набор узлов, а последний заполняется слева направо, без дырок посередине. Именно такую форму держит двоичная куча: новый элемент встаёт в первую свободную позицию последнего уровня, а при извлечении корня на его место переносится самый последний элемент, поэтому дырка внутри не появляется. Высота законченного дерева из n узлов равна $\lfloor \log_2 n \rfloor + 1$ — это наименьшая возможная высота, отсюда все оценки $O(\log n)$.

Полное (full) — у каждого узла либо ноль потомков, либо два. Про уровни здесь не сказано ничего: полное дерево может быть сколь угодно кривым, лишь бы не было узлов ровно с одним потомком.

Идеальное (perfect) — одновременно законченное и полное: все листья на одном уровне, и этот уровень заполнен целиком. Узлов ровно $n = 2^h - 1$. На собеседовании предполагать идеальность нельзя, такие деревья редки.

Из определений следует: идеальное дерево всегда законченное и всегда полное, а обратное неверно. Дерево из трёх узлов, у которого есть только левый потомок у корня, не является ни законченным, ни полным.

Термины взяты из книги Макдауэлл, раздел «Типы бинарных деревьев», — по ней идёт подготовка, поэтому в проекте они называются так же. В других русских источниках «полным» иногда называют как раз идеальное дерево; на собеседовании безопаснее уточнить, что имеется в виду, или сказать по-английски.

Отдельно: «полное представление» дерева в массиве — это про способ хранения, а не про форму, и к этим трём словам отношения не имеет. См. `TreeNotation.md`.

Где у нас: `PriorityQueue.md` — куча как законченное дерево; `BinaryTree.md` — формулы $n = 2^h - 1$ для идеального и $\lceil \log_2(n + 1) \rceil \leq h \leq n$, где левая граница достигается на законченном.